



Bitácora de Desempeño

Tercer cuatrimestre

**Asignación III. Pipeline de calidad de datos
y clasificación binaria**

Dr. **MARCOS YAMIR GOMEZ**

Jessica Melani Romero Lora

Matrícula 253220116

10 de mayo del 2026

Tabla de contenido

1. Introducción.....	3
1.1 Importación y carga de datos.....	3
2. Limpieza del separador decimal.....	3
3. Reestructuración long → wide con pivot_table.....	4
4. Diagnóstico de valores faltantes.....	4
5. Imputación con ceros.....	5
6. Creación de video_game_lover.....	5
7. Exportación.....	5
Conclusiones.....	6
Link al repositorio:.....	6

1. Introducción

Partiendo de la misma reestructuración *long* → *wide*, el enfoque central de este pipeline es el tratamiento explícito y trazable de valores faltantes. Los nulos resultantes del pivot se diagnostican con `isna().sum()` por columna y se sustituyen por cero, asumiendo que la ausencia de registro equivale a ausencia de consumo. Sobre el dataset limpio se deriva la variable binaria `video_game_lover`, que clasifica a los estudiantes con consumo de videojuegos superior a 7 horas semanales.

1.1 Importación y carga de datos

```
1  import pandas as pd
2  import numpy as np
3
4  # Importar datos
5  df = pd.read_csv("entertainment.csv")
6  print(f"[1] Shape original (long): {df.shape}")
7  print(df.head(4), "\n")
8
```

Se carga el dataset en formato *long* donde cada fila representa un par estudiante-categoría con su valor de horas semanales.

2. Limpieza del separador decimal

```
9  # Limpiar separador decimal
10 df["hours_per_week"] = (
11     df["hours_per_week"].astype(str).str.replace(",", ".").str.strip()
12 )
13 df["hours_per_week"] = pd.to_numeric(df["hours_per_week"], errors="coerce")
14
```

Se normaliza el separador decimal convirtiendo la columna a cadena de texto, reemplazando comas por puntos y eliminando espacios residuales. `pd.to_numeric(..., errors="coerce")` maneja valores no parseables convirtiéndolos en `NaN`.

3. Reestructuración long → wide con `pivot_table`

```
15 # Pivot long → wide
16 df_wide = df.pivot_table(
17     index="name",
18     columns="entertainment",
19     values="hours_per_week",
20     aggfunc="mean",
21 ).reset_index()
22 df_wide.columns.name = None
23 df_wide = df_wide[["name", "video_games", "tv_shows", "movies", "books"]]
24
```

Se obtiene una fila por estudiante con columnas independientes por categoría de entretenimiento. El agregador `aggfunc="mean"` resuelve posibles duplicados y `reset_index()` normaliza la estructura del DataFrame resultante.

4. Diagnóstico de valores faltantes

```
25 # Encontrar valores faltantes
26 print("\nValores faltantes por columna:")
27 print(df_wide.isna().sum())
28
```

Antes de cualquier imputación se ejecuta un diagnóstico explícito con `isna().sum()`, que retorna el conteo de valores `NaN` por columna. Este paso documenta el estado de calidad del dato previo a la intervención, práctica estándar en pipelines de data quality.

5. Imputación con ceros

```
30 df_wide = df_wide.fillna(0)
31 print("\nValores faltantes después de rellenar con 0's:")
32 print(df_wide.isna().sum())
33
```

Los valores nulos se sustituyen por cero mediante `fillna(0)` aplicado sobre todo el DataFrame. Esta estrategia asume semánticamente que la ausencia de registro equivale a ausencia de consumo en esa categoría. El segundo `isna().sum()` verifica que no queden nulos residuales tras la imputación.

6. Creación de `video_game_lover`

```
35 df_wide["video_game_lover"] = (df_wide["video_games"] > 7).astype(int)
36
37 print(f"\nShape final: {df_wide.shape}")
38 print(df_wide.head(5))
39 print(df_wide.describe().round(3))
40
```

Se genera una variable binaria mediante comparación booleana sobre la columna `video_games`. La expresión `df_wide["video_games"] > 7` produce una Serie de tipo `bool` que `astype(int)` convierte a enteros 0 y 1, donde 1 identifica estudiantes con consumo superior a 7 horas semanales. Esta codificación es directamente utilizable como feature en modelos de clasificación supervisada.

7. Exportación

```
41 df_wide.to_csv("entertainment_wide_img2.csv", index=False)
42 print("\n✓ Guardado: entertainment_wide_img2.csv")
```

El DataFrame procesado se exporta a CSV con `index=False` para suprimir la columna de índice en el archivo de salida, manteniendo únicamente las columnas del análisis.

Conclusiones

El pipeline establece una base sólida de calidad de datos mediante un proceso de diagnóstico e imputación explícito y reproducible. La decisión de sustituir valores faltantes por cero responde a una interpretación semántica del dato ausente: si un estudiante no registra consumo en una categoría, se asume que dicho consumo es nulo, lo cual constituye una suposición válida y conservadora en contextos de encuesta de hábitos. La variable `video_game_lover` demuestra cómo una regla de negocio simple — umbral de 7 horas semanales — puede transformarse en un indicador binario interpretable y operacionalmente útil. Su codificación como entero 0/1 la hace compatible de forma inmediata con algoritmos de clasificación supervisada, segmentación de clústeres o análisis de correlación. En conjunto, este pipeline prioriza la trazabilidad del tratamiento de datos y la generación de variables con significado analítico directo, sentando las bases para un análisis posterior robusto y reproducible.

Link al repositorio:

<https://github.com/jessicaromero-ctrl/Unsupervised-Learning/blob/main/asignacion2.py>

